



TRABALHO PRÁTICO:

Compilador em PYTHON para a linguagem Rascal

Discentes:

Vinícius Schmitz Freitas de Oliveira

RA:

109887

Docente:

Alisson Renan Svaigen

1. SUMÁRIO

1. Sumário	1
2. Introdução	2
3. Características da linguagem Rascal	2
4. Especificação	4
4.1 Especificação Léxica	
4.2 Especificação Sintática	
4.3 Especificação Semântica	
5. Decisões de Projeto	9
5.1 Funcionamento e Alterações	
5.2 Auto-avaliação do Código	
6. Visão Geral dos Módulos	14
7. Funcionamento - Compilação e Execução	15
8. Conclusão	16

2. INTRODUÇÃO

O presente trabalho propõe a construção de um compilador para a linguagem experimental Rascal (Reduces Pascal). O compilador faz as 3 principais etapas de análise de um compilador: léxica, sintática e semântica. E além disso a geração de um código intermediário para ser interpretado pela máquina MEPA

O trabalho utiliza a ferramenta auxiliar PLY e faz a implementação do compilador utilizando a linguagem python. Também é gerada explicitamente uma Árvore Sintática Abstrata (AST) a partir da análise sintática. O compilador é executado a partir da linha de comando, sendo passados como argumentos o arquivo de código fonte em Rascal e o arquivo de saída, onde será gravado o código objeto da MEPA.

3. CARACTERÍSTICAS DA LINGUAGEM RASCAL

É um subconjunto de Pascal, então compartilha a maioria das características dessa linguagem com algumas pequenas alterações, como por exemplo que Rascal é case-sensitive (diferencia caracteres maiúsculos de minúsculos).

PALAVRAS RESERVADAS

program	begin	if	do	or
var	end	then	read	not
procedure	false	else	write	div
function	true	while	and	integer
				boolean

SÍMBOLOS

(	,	-	<>	>=
)	;	*	>	<=
.	+	=	<	:=
				:

Vamos considerar que Rascal considera apenas a existência de n°s inteiros. Desse modo, assim como em Pascal a operação de **divisão inteira** é representada pela palavra **div**, então ainda temos:

div

TIPAGEM

Rascal possui apenas 2 tipos primitivos:

- **integer**: representa valores numéricos inteiros;
- **boolean**: representa valores lógicos (**false** e **true**)

Diferente de Pascal, Rascal não permite que novos tipos sejam definidos pelo usuário.

INTERFACES DE ENTRADA E SAÍDA

O programa possui dois comandos de entrada e saída: **read** e **write**:

- **read**: solicita a leitura de dados pela interface de entrada padrão,
- **write**: exibe valores na interface de saída padrão.

Eles são procedimentos pré-definidos pela linguagem e são capazes de receber um número indeterminado de argumentos do tipo integer ou boolean. No entanto, não é possível “escrever uma mensagem” associada à etapa de leitura, visto que Rascal possui apenas tipos inteiros e booleanos. Exemplos funcionais:

```
read(x, y);  
write(x, y, 2*x+y, 3);
```

IDENTIFICADORES

São **nomes associados a entidades do programa**, como:

- variáveis;
- tipos;
- sub-rotinas (funções e procedimentos);
- etc...

Estes nomes podem ser criados pelo usuários, mas também podem ser pré-definidos

4. ESPECIFICAÇÃO

Especificação Léxica:

IDENTIFICADORES

- ☑ São nomes associados a entidades do programa, como:
 - Variáveis;
 - Tipos;
 - Sub-rotinas (funções e procedimentos);
 - etc...
- ☑ Estes nomes podem ser criados pelo usuário, mas também podem ser pré-definidos pela linguagem, como os 2 tipos primitivos **integer** e **boolean** que são pré-definidos, logo são palavras reservadas e deverão ser inseridas na tabela de símbolos logo no início da fase de análise semântica/tradução.
 - Caso sejam utilizados no programa, deve-se verificar a correção de seu uso. Exemplo: o identificador **integer**, inserido na tabela de símbolos como um tipo não pode ser empregado como nome de uma variável declarada.
 - Os identificadores devem sempre iniciar com uma letra, e possuir apenas:
 - Letras minúsculas ou maiúsculas;
 - Sublinhas (underline ‘_’);
 - Dígitos (0 a 9).

NÚMEROS

- ☑ Devem ser representadas na base decimal e podem conter qualquer combinação de dígitos entre 0 a 9.
- ☑ Os números negativos só serão processados na fase sintática. Dessa forma, o número -42 por exemplo, consistirá em 2 símbolos (tokens):
 - ‘-’
 - ‘42’

LÓGICOS

- ☑ As constantes lógicas **falso** e **verdadeiro** são representadas respectivamente pelas palavras reservadas:
 - **false**
 - **true**

COMENTÁRIOS

- ☑ A linguagem Rascal não aceita inserção de comentários no código. Além de que espaços em brancos, tabulações e quebras de linhas são totalmente ignoradas.

Especificação Sintática:

REGRAS DE PRODUÇÃO

- ☑ <programa> ::= 'program' <identificador> ';' <bloco> '.'
- ☑ <bloco> ::= [<seção_declarção_variáveis>] [<seção_declarção_subrotinas>] <comando_composto>
- ☑ <seção_declarção_variáveis> ::= 'var' <declarção_variáveis> ';' { <declarção_variáveis> ';' }
- ☑ <declarção_variáveis> ::= <lista_identificadores> ':' <tipo>
- ☑ <lista_identificadores> ::= <identificador> { ',' <identificador> }
- ☑ <tipo> ::= 'boolean' | 'integer'
- ☑ <seção_declarção_subrotinas> ::= { (<declarção_procedimento> | <declarção_função>) ';' }
- ☑ <declarção_procedimento> ::= 'procedure' <identificador> [<parâmetros_formais>] ';' <bloco_subrot>
- ☑ <declarção_função> ::= 'function' <identificador> [<parâmetros_formais>] ':' <tipo> ';' <bloco_subrot>
- ☑ <bloco_subrot> ::= [<seção_declarção_variáveis>] <comando_composto>
- ☑ <parâmetros_formais> ::= '(' <declarção_parâmetros> { ',' <declarção_parâmetros> } ')'
- ☑ <declarção_parâmetros> ::= <lista_identificadores> ':' <tipo>
- ☑ <comando_composto> ::= 'begin' <comando> { ';' <comando> } 'end'
- ☑ <comando> ::= <atribuição> | <chamada_procedimento> | <condicional> | <repetição> | <leitura> | <escrita> | <comando_composto>
- ☑ <atribuição> ::= <identificador> '=' <expressão>
- ☑ <chamada_procedimento> ::= <identificador> ['(' <lista_expressões> ')'] **regra com erro**
- ☑ <chamada_procedimento> ::= <identificador> '(' [<lista_expressões>] ')' **regra corrigida**
- ☑ <condicional> ::= 'if' <expressão> 'then' <comando> ['else' <comando>]
- ☑ <repetição> ::= 'while' <expressão> 'do' <comando>
- ☑ <leitura> ::= 'read' '(' <lista_identificadores> ')'
- ☑ <escrita> ::= 'write' '(' <lista_expressões> ')'
- ☑ <lista_expressões> ::= <expressão> { ',' <expressão> }

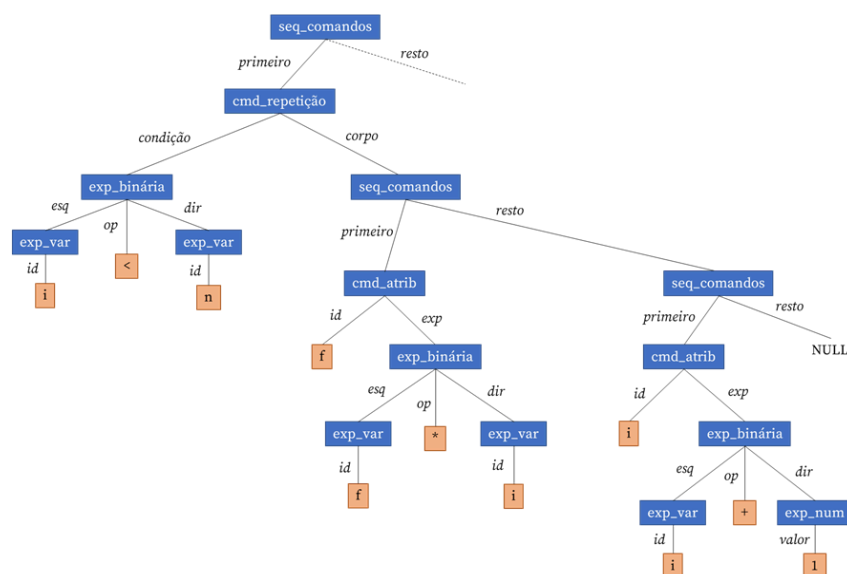
- ☑ <expressão> ::= <expressão_simples> [<relação> <expressão_simples>]
- ☑ <relação> ::= '=' | '<' | '<=' | '>' | '>='
- ☑ <expressão_simples> ::= <termo> { ('+' | '-' | 'or') <termo> }
- ☑ <termo> ::= <fator> { ('*' | 'div' | 'and') <fator> }
- ☑ <fator> ::= <variável> | <número> | <lógico> | <chamada_função> | '(' <expressão> ')' | 'not' <fator> | '-' <fator>
- ☑ <variável> ::= <identificador>
- ☑ <lógico> ::= 'false' | 'true'
- ☑ <chamada_função> ::= <identificador> ['(' <lista_expressões> ')'] **regra com erro**
- ☑ <chamada_função> ::= <identificador> '(' [<lista_expressões>] ')' **regra corrigida**

CONSTRUÇÃO DA ÁRVORE SINTÁTICA

- ☑ ~~A análise sintática faz a correção da sintaxe do programa, emitir erros apropriados e constrói uma Árvore Sintática Abstrata (AST). Ela é uma representação simplificada da estrutura do programa lido, abstraindo as derivações sintáticas em nós que representam as construções das regras de produção da linguagem. Por exemplo:~~

```
while i < n do
begin
  f := f * i;
  i := i + 1
end;
```

Este fragmento de código Rascal poderia ter sua AST construída da seguinte forma:



Especificação Semântica:

Com a AST construída, a geração de código é realizada por meio de uma travessia nela, e cada nó tem suas próprias regras de verificação semântica e geração de código. A seguir os principais tratamentos semânticos das principais construções da linguagem.

PROGRAMA

- ☐ Um programa consiste em:
 - Uma sequência de declarações de variáveis (opcional), seguida de
 - Uma sequência de declarações de sub-rotinas (opcional), e por fim
 - Uma sequência de comandos (mandatória).
- ☐ Por ser a estrutura principal do programa (nó raiz da AST), a análise começa e termina com ela. Portanto:
 - É aqui que a tabela de símbolos será inicializada e o escopo global será definido;
 - Todas as declarações realizadas no programa estão dentro do escopo global, que permanece ativo até o fim da execução;
- ☐ Cada sub-rotina define seu próprio escopo local e, como não são permitidas declarações em blocos de comandos, blocos **begin ... end** não criam novo escopo.

DECLARAÇÃO

- ☐ Declarações de variáveis, funções, procedimentos e parâmetros instalam os símbolos correspondentes e suas vinculações (tipo, escopo, posição no escopo, etc...) na tabela de símbolos.
- ☐ Antes de instalar um novo identificador, deve-se verificar se ele já existe no mesmo escopo:
 - Caso não exista, ele é instalado normalmente;
 - Caso exista, a nova declaração é desconsiderada e deve ser emitido um alerta.

COMANDOS

Os comandos que merecem verificação semântica são:

- ☐ **Atribuição:**
 - A variável à esquerda da atribuição deve estar declarada e acessível no escopo atual;
 - A expressão à direita deve ser semanticamente correta e possuir o mesmo tipo de variável à esquerda.

☐ **Condicional e repetição:**

- A expressão condicional do **if** e do **while** deve ser válida e resultar em um valor do tipo lógico.

☐ **Chamada de procedimento:**

- O procedimento deve estar declarado e visível no escopo atual;
- O n° de argumentos deve coincidir com o n° de parâmetros;
- Os tipos dos argumentos devem corresponder, na mesma ordem, aos tipos dos parâmetros;
- Todos os parâmetros são passados por valor, sendo assim, os argumentos reais são avaliados e copiados para os parâmetros formais.

☐ **Retorno de função:**

- Toda função deve retornar exatamente um valor;
- O tipo do valor retornado deve coincidir com o tipo declarado na função;
- O retorno é realizado por meio de atribuição ao próprio identificador da função dentro de seu corpo;
- A ausência de retorno, ou retorno de tipo incompatível, deve gerar erro semântico.

☐ **Leitura (read):**

- Os argumentos devem ser variáveis declaradas e visíveis no escopo atual.

☐ **Escrita (write):**

- Os argumentos devem ser expressões válidas e bem tipadas.

☐ **Expressões:**

- Aritmética: o(s) operando(s) devem ser do tipo inteiro. O tipo resultante é inteiro;
- Relacional: os operandos devem ser do tipo inteiro. O tipo resultante é lógico;
- Igualdade/Diferença: os operandos devem ser do mesmo tipo primitivo. O tipo resultante é lógico;
- Lógica: o(s) operando(s) devem ser do tipo lógico. O tipo resultante é lógico;
- Uso de variável: a variável deve estar declarada e visível no escopo atual. O tipo resultante do uso é o tipo declarado para a variável;
- Chamada de função: análise análoga à chamada de procedimento. O tipo resultante da chamada é o tipo declarado para a função.

5. DECISÕES DE PROJETO E IMPLEMENTAÇÃO

O compilador da linguagem Rascal foi implementado com base no código do compilador para a linguagem CALCULADin, que se trata de uma linguagem de programação experimental assim como Rascal porém cujo foco é realizar operações aritméticas básicas de números reais (+ - * /), além de permitir a análise de condições lógicas (true ou false). As principais mudanças e acréscimos para a implementação do compilador para a linguagem Rascal foram:

Analizador Léxico:

Para as **palavras reservadas** as alterações realizadas estão em **amarelo** e o que foi acrescentado em **verde**:

<i>Palavras reservadas</i>	<i>RASCAL</i>	<i>TOKEN CALCULADIN</i>	<i>TOKEN RASCAL</i>
<i>Geral</i>	<i>program</i> <i>var</i> <i>procedure</i> <i>function</i> <i>begin</i> <i>end</i> <i>read</i> <i>write</i> <i>div</i> <i>do</i>	<i>CALCULADIN</i> - - - <i>INICIAR</i> <i>FINALIZAR</i> <i>INPUT</i> <i>OUTPUT</i> <i>DIVIDE</i> -	PROGRAM VAR PROCEDURE FUNCTION BEGIN END READ WRITE (nos símbolos) DO
<i>Tipos</i>	<i>integer</i> <i>boolean</i>	<i>REAL_TIPO</i> <i>BOOL</i>	INTEGER BOOLEAN
<i>Literais boolean</i>	<i>false</i> <i>true</i>	<i>FALSE</i> <i>TRUE</i>	<i>FALSE</i> <i>TRUE</i>
<i>Condicionais</i>	<i>if</i> <i>then</i> <i>else</i> <i>while</i>	<i>IF</i> - <i>ELSE</i> -	<i>IF</i> THEN <i>ELSE</i> WHILE
<i>Operadores lógicos</i>	<i>and</i> <i>or</i> <i>not</i>	<i>AND</i> <i>OR</i> <i>NOT</i>	<i>AND</i> <i>OR</i> <i>NOT</i>
<i>Divisão</i>	<i>div</i>	-	DIVIDE

E para os símbolos da linguagem, incluindo palavra reservada *div* que em Rascal é utilizada para a divisão ao invés do símbolo /:

<i>Símbolo</i>	<i>RASCAL</i>	<i>TOKEN CALCULADIN</i>	<i>TOKEN RASCAL</i>
<i>Operadores aritméticos</i>	+ - * <i>div</i>	PLUS MINUS TIMES DIVIDE (/)	PLUS MINUS TIMES DIVIDE
<i>Operadores Relacionais e lógicos</i>	:= = <> < > <= >=	- EQUALS (atrib) NOT_EQ (!=) LESS GREATER LESS_EQ GREATER_EQ	ATRIB EQUAL (op log) NOT_EQ LESS GREATER LESS_EQ GREATER_EQ
<i>Delimitadores</i>	() ; : . ,	LPAREN RPAREN SEMICOL COLON PERIOD -	LPAREN RPAREN SEMICOL COLON PERIOD COMMA

Sendo assim, as principais mudanças nos token para o analisador léxico funcionar corretamente para a linguagem Rascal foram:

PALAVRAS RESERVADAS

Para permitir depois com as regras de produção a declaração de diferentes nomes de programa tal como declaração de variáveis:

- Alteração de **CALCULADIN** → **PROGRAM** ;
- Acréscimo de **VAR** ;

Para permitir declaração de subrotinas:

- Acréscimo de **PROCEDURE** ;
- Acréscimo de **FUNCTION** ;

Alteração de nome da palavra reservada apenas:

- Alteração de **INICAR** → **BEGIN** ;
- Alteração de **FINALIZAR** → **END** ;
- Alteração de **INPUT** → **READ** ;
- Alteração de **OUTPUT** → **WRITE** ;

Para os tipos:

- Alteração de **REAL_TIPO** → **INTEGER**, pois Rascal trabalha apenas com números inteiros ;
- Alteração de **BOOL** → **BOOLEAN** ;

Para condicional:

- Acréscimo de **THEN** ;

Para operação de divisão:

- Acréscimo de **div** (sendo uma palavra reservada para representar um símbolo de operação aritmética ;

Para laços de repetição:

- Acréscimo de **WHILE** ;
- Acréscimo de **DO** ;

Para operação de divisão:

- Acréscimo de **DIVIDE** para **div**.

SÍMBOLOS

Alteração de símbolo para a divisão, porém o token continua o mesmo:

- Alteração do símbolo ‘/’ para ‘**div**’ (token continua sendo **DIVIDE**) ;

Alteração e acréscimos em operadores relacionais e lógicos:

- Alteração do símbolo ‘=’ → ‘:=’ para representar a operação relacional de atribuição, bem como alteração do token **EQUALS** → **ATRIB** ;
- Assim, acréscimo do símbolo ‘=’ para agora ser utilizado para operação lógica de igualdade com token **EQUAL**, antes sendo **EQUALS** token de atribuição na linguagem CALCULADin (enquanto para operação lógica de igualdade era utilizado o símbolo ‘==’ com token **EQ_EQ**, símbolo removido do programa para Rascal) ;
- Bem como acréscimo do símbolo ‘<>’ para operação lógica de desigualdade com token **NOT_EQ**, mesmo token utilizado em CALCULADin (porém anteriormente com símbolo ‘!=’, também removido do programa atual).
- Por fim, acréscimo de ‘,’ com token **COMMA** para múltiplas variáveis e listas

EXPRESSÕES REGULARES PARA TOKENS MAIS COMPLEXOS

Para formação de um número foi feita a seguinte alteração, alterando formação de números reais para apenas números inteiros:

- **r'** \d+\.\d+|\d+' → **r'** \d+' ;

Para formação de identificador a seguinte alteração no código foi feita, permitindo letras maiúsculas e o caractere especial ‘_’ na declaração, e mantendo a regra de que não é permitido iniciar com n° e ‘_’:

- `r' [a-z] [a-z0-9]*' → r' [A-Za-z] [A-Za-z0-9_]*'`

Analizador Sintático (*Parser*):

Para implementar as regras lógicas incluindo a função das notações { } e [] indicando repetição e opção respectivamente, foram adicionados alguns módulos a mais no código denominados como “lista”, para satisfazer esses funcionamentos das regras:

- `<programa> ::= 'program' <identificador> ';' <bloco> '!'`
 - o ***def p_PROGRAMA(p):***
- `<bloco> ::= [ <seção_declaração_variáveis> ] [ <seção_declaração_subrotinas> ] <comando_composto>`
 - o ***def p_BLOCO(p):***
- `<declaração_variáveis> ::= <lista_identificadores> ':' <tipo>`
 - o ***def p_SECAO_DECLARACAO_VARIAVEIS(p):***
 - o ***def p_LISTA_DECL_VAR(p):***
 - o ***def p_DECL_VAR(p):***
- `<lista_identificadores> ::= <identificador> { ',' <identificador> }`
 - o ***def p_LISTA_ID(p):***
- `<tipo> ::= 'boolean' | 'integer'`
 - o ***def p_TIPO(p):***
- `<seção_declaração_subrotinas> ::= { ( <declaração_procedimento> | <declaração_função> ) ';' }`
 - o ***def p_SECAO_DECL_SUBROT(p):***
 - o ***def p_LISTA_DECL_SUBROT(p):***
 - o ***def p_DECL_SUBROT(p):***
- `<declaração_procedimento> ::= 'procedure' <identificador> [ <parâmetros_formais> ] ';' <bloco_subrot>`
 - o ***def p_DECL_PROC(p):***
- `<declaração_função> ::= 'function' <identificador> [ <parâmetros_formais> ] ':' <tipo> ';' <bloco_subrot>`
 - o ***def p_DECL_FUNC(p):***
- `<bloco_subrot> ::= [ <seção_declaração_variáveis> ] <comando_composto>`
 - o ***def p_BLOCO_SUBROT(p):***
- `<parâmetros_formais> ::= '(' <declaração_parâmetros> { ';' <declaração_parâmetros> } ')'`
 - o ***def p_PARAMETROS_FORMAIS(p):***

- *def p_LISTA_PARAMETROS_FORMAIS(p):*
- <declaração_parâmetros> ::= <lista_identificadores> ':' <tipo>
- *def p_DECL_PARAMETROS(p):*
- <comando_composto> ::= 'begin' <comando> { ';' <comando> } 'end'
- *def p_COMANDO_COMPOSTO(p):*
 - *def p_LISTA_COMANDO_COMPOSTO(p):*
- <comando> ::= <atribuição> | <chamada_procedimento> | <condicional> | <repetição> | <leitura> | <escrita> | <comando_composto>
- *def p_COMANDO(p):*
- <atribuição> ::= <identificador> ':=' <expressão>
- *def p_ATRIBUICAO(p):*
- <chamada_procedimento> ::= <identificador> '(' [<lista_expressões>] ')' **regra corrigida**
- *def p_CHAMADA_PROC(p):*
- <condicional> ::= 'if' <expressão> 'then' <comando> ['else' <comando>]
- *def p_CONDICIONAL(p):*
- <repetição> ::= 'while' <expressão> 'do' <comando>
- *def p_REPETICAO(p):*
- <leitura> ::= 'read' '(' <lista_identificadores> ')'
- *def p_LEITURA(p):*
- <escrita> ::= 'write' '(' <lista_expressões> ')'
- *def p_ESCRITA(p):*
- <lista_expressões> ::= <expressão> { ',' <expressão> }
- *def p_LISTA_EXPRESSOES(p):*
- <expressão> ::= <expressão_simples> [<relação> <expressão_simples>]
- *def p_EXPRESSAO(p):*
- <relação> ::= '=' | '<' | '<=' | '>' | '>='
- *def p_RELACAO(p):*
- <expressão_simples> ::= <termo> { ('+' | '-' | 'or') <termo> }
- *def p_EXPRESSAO_SIMPLES(p):*
 - *def p_LISTA_EXPRESSAO_SIMPLES(p):*
- <termo> ::= <fator> { ('*' | 'div' | 'and') <fator> }
- *def p_TERMOS(p):*
 - *def p_LISTA_TERMOS(p):*
- <fator> ::= <variável> | <número> | <lógico> | <chamada_função> | '(' <expressão> ')' | 'not' <fator> | '-' <fator>
- *def p_FATOR(p):*
- <variável> ::= <identificador>
- *def p_VARIABEL(p):*
- <lógico> ::= 'false' | 'true'
- *def p_LOGICO(p):*
- <chamada_função> ::= <identificador> '(' [<lista_expressões>] ')' **regra corrigida**
- *def p_CHAMADA_FUNCTION(p):*

6. VISÃO GERAL DOS MÓDULOS

O compilador foi estruturado em dois módulos principais: **analisador léxico** e **analisador sintático**, ambos baseados na biblioteca **PLY**, que fornece as ferramentas **lex** e **yacc** para implementação de compiladores em Python.

1. Módulo `lex_rascal.py` (Analisador Léxico)

Responsável por transformar o código-fonte em uma sequência de tokens.

Funções principais:

- Definição das **palavras reservadas**, **símbolos**, **operadores**, **delimitadores** e **tipos** da linguagem.
- Regras de **expressões regulares** para identificação de identificadores, números e operadores.
- Tratamento de erros léxicos (caracteres ilegais, identificadores inválidos, etc).
- Implementação da função `lexer = lex.lex()`, que inicializa o analisador.

Este módulo garante que apenas cadeias válidas cheguem à etapa sintática.

2. Módulo `parser_rascal2.py` (Analisador Sintático + AST)

Responsável por reconhecer a estrutura da linguagem e construir a **Árvore Sintática Abstrata (AST)**.

Componentes principais:

- Especificação completa das **regras de produção** da linguagem Rascal.
- Criação explícita da AST em cada regra, armazenando nós e filhos.
- Verificações semânticas básicas diretamente no parser (estrutura e coerência geral das construções).
- Tratamento de **erros sintáticos**, exibindo mensagem clara, com tipo do token, valor e posição.

O módulo é executado carregando o analisador léxico e, ao final, imprimindo a AST e a tabela de símbolos (quando existente).

7. FUNCIONAMENTO - COMPILAÇÃO E EXECUÇÃO

O compilador desenvolvido funciona como um **pipeline clássico** com duas etapas principais: análise léxica e análise sintática.

1. Execução via Linha de Comando

O compilador é executado passando-se o programa Rascal como argumento:

```
python parser_rascal2.py exemplo.ras
```

O fluxo básico é o seguinte:

2. Etapas de Funcionamento

(a) Análise Léxica

- O arquivo-fonte é lido.
- Cada caractere é classificado conforme as regras definidas em `lex_rascal.py`.
- São produzidos tokens como `ID`, `NUM`, `PLUS`, `BEGIN`, `WRITE`, etc.
- Tokens inválidos geram mensagens imediatas.

(b) Análise Sintática

- A lista de tokens é enviada ao parser criado com `yacc.yacc()`.
- O parser aplica as regras definidas, construindo a AST.
- Se tudo estiver correto, exibe:
 - Confirmação da análise
 - A árvore sintática abstrata
 - A tabela de símbolos (quando gerada)

(c) Falhas

- Erros léxicos interrompem o processo no início.
- Erros sintáticos exibem o token inesperado e encerram o parsing.

3. Resultado Final

Como o projeto implementa apenas as fases de **análise** (léxica e sintática), o compilador não gera código MEPA nem executa o programa — ele apenas confirma se o código Rascal está correto, mostrando a AST produzida.

8. CONCLUSÃO

O trabalho atinge o objetivo essencial: construir um compilador funcional para a linguagem Rascal contendo análise léxica completa, análise sintática conforme a gramática oficial e uma AST totalmente navegável. A estrutura modular torna o código simples de expandir, permitindo inserir comportamentos mais avançados futuramente.

Contudo, **duas etapas não foram implementadas**:

- **Geração de código intermediário MEPA**
- **Analisador semântico completo**

A razão é simples: essas fases exigem uma quantidade significativa de regras adicionais (tipagem formal, verificação de escopos, coerência de parâmetros, retorno de funções, alocação de endereços, emissões de instruções MEPA, gerenciamento de pilha, etc.). Cada construção da linguagem precisaria de uma rotina específica de tradução. Isso ampliaria o escopo do trabalho para além do objetivo original do projeto e demandaria mais tempo de engenharia do que o disponível para a entrega.

Assim, o foco foi garantir um compilador **correto, estável e modular** para as etapas iniciais, deixando a geração de código MEPA e a semântica profunda como etapas futuras, prontas para serem acopladas sem necessidade de reescrever o compilador inteiro.